Software Engineering — Project Plan

SIX-MEMBER TEAM • SUBMISSION VERSION

COVER PAGE

Course: ISE304 — Software Engineering
Report name: Project Plan
Project title: ITU Compass — Personalized Academic Assistant for Freshmen
Group name / number: ITU Compass
Group members: Emre Erbek 150230205, Aykut Engür 150230215, Efe Sürücü 150240225, Konuralp Polat 150230212, Samet Sav 150220209, Yeşim Gündoğdu 150220217
Instructor / Product Owner: Abdullah Cihan Ak
Submission deadline: 2026-11-03, 23:59 (Ninova deadline)
Document version: 1.1

1. INTRODUCTION

1.1 Scope

Project Overview:
ITU Compass is a personalized academic assistance platform designed primarily for freshmen and undergraduate students at Istanbul Technical University (ITU). The project aims to address the difficulty students face when accessing academic information scattered across multiple university systems, such as Kepler, Ninova, the ITU Portal, departmental websites, and official regulations.

The proposed solution is a browser-based, responsive web application (not a native mobile app) that combines a Retrieval-Augmented Generation (RAG) AI assistant with a deterministic academic rule engine. It will provide personalized answers based on students' academic profiles and official ITU sources, including verifiable references.

The platform will support students in course selection, prerequisite checking, graduation planning, internship procedures, Erasmus opportunities, and other academic processes. It will also help students seeking a thesis or graduation-project supervisor identify faculty members whose academic background and research interests align with their proposed topics. It will serve as an academic decision support tool rather than replacing existing university systems.
Scope change since the Project Charter: Following customer feedback, thesis supervisor/ project matching was added to the MVP using public faculty profiles.
In Scope:
• Authentication and Authorization: Secure application-managed registration and login using test/demo accounts, hashed passwords, JWT-based session management, and role-based

access for students and administrators. The MVP will not collect or validate ITU SSO passwords; its authentication architecture will allow a future authorized SSO integration.
• Student Profile Management: Storing department, entry year, class level, completed courses, and conversation history to provide personalized assistance.
• AI Chatbot and RAG: Answering academic questions using official ITU documents and regulations, with source links and relevant PDF page references.
• Knowledge Base Management: Processing, storing, and updating academic documents while verifying their relevance and validity.
• Academic Planner and Rule Engine: Checking course prerequisites, supporting course selection, and evaluating what-if scenarios based on academic rules.
• Graduation Progress Tracking: Identifying completed and remaining graduation requirements, including compulsory courses, electives, and credits.
• Thesis Supervisor and Project Matching: Students enter their research interests, proposed thesis topics, or project keywords. The system compares these interests against a curated set of faculty academic backgrounds, research areas, publications, and previous projects from publicly accessible official profiles (e.g. ITU Akademi), then ranks relevant potential supervisors with explainable recommendations and source links. Matches are advisory only and do not indicate a professor's approval or availability.
• Personalized Dashboard: Displaying academic deadlines, course plans, saved information, and previous conversations.
• Admin Panel: Allowing administrators to upload and update academic documents and curriculum information.
• Testing and Deployment: Developing REST APIs, implementing automated testing and CI/CD workflows, and deploying the application to a cloud platform.

Optional Scope:
• Official ITU Single Sign-On (SSO) integration may be considered in a later phase only if ITU's IT department provides formal authorization, registration details, and a supported integration method. It is not required for the MVP.

Out of Scope:
• Replacing or modifying existing ITU systems such as Kepler, Ninova, or the ITU Portal.
• Direct integration with official ITU systems or real ITU SSO authentication in the MVP; the platform will not request, store, or validate students' actual ITU passwords.
• Performing official university procedures on behalf of students, including course registration, petitions, grade appeals, or Erasmus applications.
• Supporting every ITU department and curriculum in the initial MVP.
• Providing services for graduate (MSc/PhD) students.
• Developing native iOS or Android mobile applications or distributing the platform through mobile app stores. Users will access the platform via a web browser.
• Implementing real-time synchronization with university information systems.
• Providing guaranteed or officially binding academic advising.
• Automatically assigning thesis supervisors, submitting formal supervisor requests, or claiming that recommended faculty members have available supervision capacity.
Assumptions and Constraints:
• The initial MVP will support a limited number of departments and curricula, starting with the team's own department.
• Students will manually enter their academic information, including completed courses, rather than retrieving it directly from university systems.

• The platform will depend on the availability and accuracy of official ITU documents and academic regulations.
• The supervisor-matching MVP will cover a limited, curated subset of publicly available faculty profiles. Profile data will be attributed to its source and reviewed for accuracy; automated access or integration will require compliance with applicable site terms and permissions.
• Authentication will use locally managed demo/test users; accessing real ITU SSO will remain dependent on formal institutional approval.
• The AI assistant will provide source references and explicitly indicate when reliable information is insufficient.
• The application will be developed using the technology stack defined in the Project Charter, including Python/FastAPI, PostgreSQL, React/Next.js, and RAG-related technologies.
• The project will be developed by a six-member software engineering team within the course schedule, with the final codebase due on December 27, 2026, and the demonstration scheduled for December 30, 2026.
• The platform will be implemented as a responsive web application accessible through desktop and mobile web browsers; no separate native mobile application will be developed. It will not replace official academic advising or university administrative systems.

1.2 Deliverables

The deliverables below are software product components, not the interim reports submitted during the course. Technical documentation is a deliverable in its own right.

Deliverable ID	**Deliverable name**	**Description / contents**	**Acceptance criterion**
D-01	Core Backend & Database	FastAPI backend, PostgreSQL database, REST API structure, and platform data models.	Backend starts successfully, database connection works, and core API endpoints return expected responses.
D-02	Authentication & Student Profile System	Registration, test-account login, JWT authentication, student/admin access roles, and student academic profiles. No real ITU SSO integration.	Students can register and log in using application-managed test accounts, edit profiles, and access only authorized features.
D-03	AI Chatbot & Academic Knowledge Base	RAG-based assistant using official ITU documents, document processing, vector search, and verifiable citations.	Chatbot answers supported academic questions with relevant official sources and explains when reliable information is insufficient.

D-04	Academic Planner & Graduation Tracker	Prerequisite checks, academic rule engine, course planning, graduation progress, and what-if scenarios.	System correctly checks sample course prerequisites, remaining graduation requirements, and predefined what-if cases.
D-05	Thesis Supervisor & Project Matching	Faculty recommendation system comparing student topics and research interests against a curated set of faculty academic profiles and research areas.	System recommends relevant faculty from the available dataset and gives understandable matching explanations with public source references; it does not assign supervisors.
D-06	Student Dashboard & Admin Interface	Responsive browser-based UI for chatbot, planner, progress tracker, faculty matches, and admin document management. No native mobile app.	Main features work in desktop and mobile browsers; admin users can manage supported documents and academic data.
D-07	Integrated & Tested Web Application	Frontend, backend, database and AI integration, automated tests, CI/CD workflows, and cloud deployment.	Deployed web MVP is accessible and passes agreed core functional, integration, and end-to-end tests.
D-08	Technical Documentation	Architecture overview, database schema, API documentation, setup/deployment instructions, and user guidance.	Documentation enables the team or evaluator to set up the system, understand major components, and use key features.

1.3 Functional Requirements / Tasks

Each functional task is identified by a unique T-## ID and mapped to one or more deliverables from Section 1.2. The tasks define expected system functionality; their implementation activities and effort estimates are detailed later in the WBS.

Task ID	Functional requirement / task	Description	Related deliverable ID(s)
T-01	Backend API & Database Setup	Develop REST API infrastructure, PostgreSQL database models, and data persistence for core application entities.	D-01
T-02	User Registration & Authentication	Implement registration, login, password hashing, and JWT-based authentication using application-managed test accounts.	D-01, D-02
T-03	Role-Based Access Control	Implement student and administrator roles with appropriate permissions for protected application features.	D-02
T-04	Student Profile Management	Allow students to enter and update their department, entry year, academic level, and completed courses.	D-02
T-05	Academic Document Processing	Allow authorized administrators to upload official academic documents and process them into a searchable knowledge base.	D-03, D-06
T-06	AI Academic Question Answering	Implement a RAG-based chatbot that answers questions using official ITU sources provides citations and PDF page references, and identifies insufficient evidence.	D-03
T-07	Conversation History	Store and retrieve previous chatbot conversations to support personalized interactions.	D-02, D-03
T-08	Curriculum & Prerequisite Management	Store supported curricula and course prerequisite relationships and validate course eligibility through deterministic rules.	D-04
T-09	Course Planning & What-If Analysis	Generate course-planning guidance and evaluate hypothetical scenarios, such as failing or retaking a course.	D-04
T-10	Graduation Progress Tracking	Calculate completed and remaining academic requirements based on manually entered student records.	D-04
T-11	Faculty Academic Profile Management	Collect and organize publicly available faculty research interests, publications, and project information for a limited dataset.	D-05
T-12	Thesis Supervisor & Project Matching	Compare students' research interests and proposed topics with faculty academic profiles and generate ranked, explainable recommendations with source references.	D-05
T-13	Student Dashboard	Develop a dashboard displaying academic progress, deadlines, course plans, saved information, and conversation history.	D-06

T-14	Admin Management Interface	Develop an interface for authorized administrators to maintain academic documents, curricula, and supported reference data.	D-03, D-04, D-06
T-15	Responsive Web Interface	Build and connect browser-based interfaces for the chatbot, planner, graduation tracker, and faculty matching system, supporting desktop and mobile browsers.	D-06
T-16	System Integration, Testing & Deployment	Integrate application components, implement automated tests and CI/CD workflows, and deploy the MVP to a cloud platform.	D-07
T-17	Technical Documentation	Prepare system architecture, database, API, installation, deployment, and user documentation.	D-08

1.4 Non-functional Issues

The following non-functional requirements describe the expected quality of the ITU Compass web application and how the MVP will be evaluated. Numerical targets below are proposed acceptance targets for team review, not verified university requirements. Testing will use a limited dataset and demo accounts.

Quality attribute	**Requirement / concern**	**How it will be assessed**
Security	Use hashed passwords, JWT validation, and student/admin role-based access. The MVP must not collect or validate actual ITU account passwords.	Test invalid logins, expired tokens, blocked student access to admin APIs, and secure password storage
Privacy & data protection	Keep manually entered academic profiles and chat histories separated by student account; avoid exposing user data or secrets in API responses and logs.	Attempt cross-account access and review sample API responses, database fields, and logs for unintended disclosure.
Performance	Proposed MVP targets: 95% of routine non-AI API requests within 2 seconds for 10 concurrent demo users; 90% of chatbot replies within 15 seconds when the LLM service is available.	Run repeatable load tests and timed sample questions; record p95 API latency and p90 chatbot response time.
Usability	Provide clear navigation for the chatbot, course planner, graduation tracker, and faculty matching through a responsive web interface.	Ask five student testers to complete profile update, academic question, and course-planning tasks; target at least four successful unaided completions.

Reliability & error handling	Handle invalid input, missing official sources, document-processing errors, and AI-service interruptions without crashes or misleading academic advice.	Test invalid uploads, unavailable LLM responses, missing knowledge-base evidence, and API errors; verify useful error messages and safe fallback behavior.
Academic accuracy & traceability	Ground AI guidance in official ITU sources with links and relevant PDF pages; evaluate prerequisite and graduation rules deterministically. Faculty matches must be explained and not imply acceptance.	Review a curated set of 20 academic questions, 10 rule-engine scenarios, and five faculty-matching examples; check citations, rule outcomes, and honest uncertainty handling.
Browser compatibility & accessibility	Support recent desktop browsers and responsive mobile web browsing without developing native mobile apps; ensure legible layouts and keyboard-accessible main controls.	Check Chrome, Firefox, and Safari plus representative 375 px, 768 px, and desktop layouts; perform keyboard navigation and readability checks.
Maintainability & testability	Use modular FastAPI/Next.js code, database migrations, documented configuration, and automated testing through GitHub Actions.	Run Pytest, frontend component tests, and core end-to-end checks in CI; confirm the project can be set up from documented steps.

2. PROJECT PLAN — WORK BREAKDOWN STRUCTURE (WBS)

The following work breakdown decomposes all 17 functional tasks into implementable and verifiable activities. Size is a preliminary relative complexity estimate: T = Tiny, S = Small, M = Medium, L = Large, XL = Extra-large; person-week estimates will be calculated separately in Section 3.

WBS activity ID	**Parent task ID**	**Activity / subtask**	**Description / completion condition**	**Size (T/S/ M/L/ XL)**
WBS-1.1	T-01	Design database schema	Define user, course, curriculum, chat, document and faculty models; migration plan reviewed.	M
WBS-1.2	T-01	Build core API and persistence	Set up FastAPI, PostgreSQL connection, migrations and basic CRUD endpoints; smoke tests pass.	L
WBS-2.1	T-02	Register users securely	Create demo-account registration and password hashing; valid users can register.	M
WBS-2.2	T-02	Implement JWT login	Provide token issuance, expiry and login/ logout handling; authentication tests pass	M

WBS-3.1	T-03	Define access roles	Document student/admin permissions for endpoints and admin features; matrix approved.	S
WBS-3.2	T-03	Enforce protected access	Add role guards on protected routes; forbidden requests are rejected in tests.	M
WBS-4.1	T-04	Implement academic profile	Build endpoints for department, entry yea and class level; updates persist.	M
WBS-4.2	T-04	Record completed courses	Store and validate manually entered completed courses and grades; profile tests pass.	M
WBS-5.1	T-05	Curate official ITU sources	Select initial regulations, curricula and calendar PDFs; record URLs and versions.	M
WBS-5.2	T-05	Index academic documents	Extract PDF text and page metadata, chunk and embed into pgvector; retrieval works.	L
WBS-5.3	T-05	Implement document updates	Create secure upload/re-index backend flow; changed documents are searchable.	M
WBS-6.1	T-06	Build grounded RAG pipeline	Connect retrieval to LLM generation and user context; sample queries return grounded answers.	XL
WBS-6.2	T-06	Cite sources and handle gaps	Show document links and page numbers; respond safely when sources are insufficient.	M
WBS-6.3	T-06	Evaluate chatbot responses	Test sample ITU questions, inspect citations and refine retrieval; findings recorded.	M
WBS-7.1	T-07	Persist chat sessions	Create per-user conversation and message storage; new messages survive reload.	M
WBS-7.2	T-07	Restore conversation context	Implement history listing and bounded context retrieval with account isolation.	M
WBS-8.1	T-08	Model supported curriculum	Enter selected department courses, credits and curriculum versions; dataset reviewed.	M
WBS-8.2	T-08	Build prerequisite rule graph	Implement NetworkX prerequisite checks and test eligibility cases against examples.	L
WBS-9.1	T-09	Generate course guidance	Evaluate eligible courses using completed records and curriculum rules; results displayed.	M
WBS-9.2	T-09	Simulate academic scenarios	Compute fail/retake what-if outcomes and explain rule-based changes; cases tested	L

WBS-10.1	T-10	Define graduation criteria	Model required/elective courses and credit thresholds for MVP curricula; rules reviewed.	M
WBS-10.2	T-10	Calculate completion gaps	Show remaining required courses, electives and credits; sample transcripts validated.	L
WBS-11.1	T-11	Curate faculty profiles	Select public ITU academic profiles and capture research areas with source references.	M
WBS-11.2	T-11	Normalize faculty metadata	Store interests, selected publications and projects with dates/attribution; records searchable.	M
WBS-12.1	T-12	Implement matching and ranking	Compare student topics with faculty metadata using a keyword baseline; allow semantic extension.	L
WBS-12.2	T-12	Explain recommendations	Return ranked faculty with research-match reasons and profile links; no availability claim.	M
WBS-13.1	T-13	Design student dashboard	Create responsive layout and reusable progress, reminder and history components.	M
WBS-13.2	T-13	Connect dashboard data	Populate academic status, deadlines, saved items and chats from APIs; views tested.	M
WBS-14.1	T-14	Build admin management views	Create document, faculty and curriculum maintenance screens using existing APIs.	M
WBS-14.2	T-14	Validate administrator actions	Enforce admin-only operations, input checks and update feedback; tests pass.	M
WBS-15.1	T-15	Build core responsive pages	Implement chat, planner, graduation and matching views for desktop/mobile browsers.	L
WBS-15.2	T-15	Connect frontend to APIs	Wire auth, loading, error states and feature requests; end-to-end flows work.	L
WBS-16.1	T-16	Integrate and test components	Connect backend, frontend, DB and AI flows; run automated functional/ integration tests.	L
WBS-16.2	T-16	Configure CI and hosting	Set up GitHub Actions, secrets/config and cloud deployment; pipeline runs successfully.	M
WBS-16.3	T-16	Run MVP acceptance tests	Exercise key scenarios, fix blocking defects and verify deployed demonstration.	L
WBS-17.1	T-17	Document architecture and APIs	Write architecture, DB model, endpoint and setup/deployment guidance; team review complete.	M

WBS-17.2	T-17	Prepare user instructions	Write short student/admin usage guide and demo instructions; steps verified.	S

WBS decomposition notes:
The WBS follows the functional task IDs from Section 1.3. Foundational database/API work supports authentication, the rule engine, RAG, and the web UI. Official ITU documents and a limited set of faculty profiles must be curated before retrieval and matching can be evaluated. Curriculum rules depend on the selected department and student-entered course records. UI integration depends on stable APIs; end-to-end testing and deployment depend on integrated components. Test activities are included in each workstream and consolidated under T-16. The matching algorithm may start with a keyword baseline and later use semantic embeddings if feasible; this design choice is still subject to team review. Sizes are proposals to be validated with team availability and dependencies in Sections 3–5.

Test-first implementation and verification workflow (planned):
For each functional slice, the owner will derive testable examples from the evolving requirements and prepare unit, API, or component checks before or alongside implementation. Work will proceed in short implement–test–refactor iterations, with automated tests run locally and through GitHub Actions on pull requests. Prerequisite, graduation, and authorization logic will use positive, negative, and boundary-condition test fixtures. RAG and faculty matching will use a curated set of academic questions/topics; source attribution, citation correctness, and potentially unsupported answers will be evaluated with human review. Each test case and its outcome will be linked to the relevant GitHub Issue/pull request and WBS ID. Task-level checks are included in Section 3 effort estimates; cross-component and final acceptance testing remain under WBS-16.x.

Course guideline: User stories for these activities belong in the separate requirements document; this project plan identifies and sizes implementation activities.

3. ESTIMATES

The effort estimates below are preliminary person-weeks (one person-week = 40 hours of individual effort, which can be distributed across all seven days of a calendar week). They use the relative sizes established in Section 2 and assume a limited-department web MVP, application-managed test accounts, and a curated set of ITU documents and faculty profiles.

WBS activity ID	Size	Estimated effort (man-weeks)	Basis / assumptions
WBS-1.1	M	0.30	Several linked domain models and an initial migration review.
WBS-1.2	L	0.65	API skeleton, DB connection, migrations and CRUD smoke tests.
WBS-2.1	M	0.30	Registration validation, hashing and account-creation tests.

WBS-2.2	M	0.30	JWT issuance, expiry, login/logout endpoints and tests.
WBS-3.1	S	0.15	Short permission matrix plus review with the team.
WBS-3.2	M	0.30	Role guards across protected endpoints and rejection tests.
WBS-4.1	M	0.30	Profile API fields, persistence and input validation.
WBS-4.2	M	0.30	Completed-course/grade storage and validation tests.
WBS-5.1	M	0.30	Select initial official PDFs; verify source URLs and versions.
WBS-5.2	L	0.65	PDF parsing, page metadata, embeddings and vector indexing.
WBS-5.3	M	0.30	Secure upload/re-index flow and update verification.
WBS-6.1	XL	1.10	Retrieval, LLM generation and user context must work together.
WBS-6.2	M	0.30	Page-level citations, source links and no-evidence fallbacks.
WBS-6.3	M	0.30	Sample academic questions, citation checks and refinement.
WBS-7.1	M	0.30	Per-user session/message tables and persistence endpoints.
WBS-7.2	M	0.30	History listing, context limits and account-isolation tests.
WBS-8.1	M	0.30	Enter selected curriculum and verify credits/ versions.
WBS-8.2	L	0.65	NetworkX graph logic and prerequisite unit-test cases.
WBS-9.1	M	0.30	Eligibility rules and explanations from student records.
WBS-9.2	L	0.65	Failed/retaken-course branching with outcome verification.
WBS-10.1	M	0.30	Define required/elective and credit rules for initial curricula.
WBS-10.2	L	0.65	Remaining-credit calculations and sample transcript checks.
WBS-11.1	M	0.30	Curate a limited, attributed set of public faculty profiles.
WBS-11.2	M	0.30	Normalize interests, selected publications and storage.
WBS-12.1	L	0.65	Implement keyword matching baseline and ranking checks.

WBS-12.2	M	0.30	Matching explanations, links and supervisor disclaimer.
WBS-13.1	M	0.30	Reusable responsive dashboard components and layout.
WBS-13.2	M	0.30	Connect dashboard widgets to data APIs and test views.
WBS-14.1	M	0.30	Document, faculty and curriculum admin forms.
WBS-14.2	M	0.30	Permission checks, validation and admin error feedback.
WBS-15.1	L	0.65	Core chat/planner/tracker/matching pages across breakpoints.
WBS-15.2	L	0.65	Auth, request states and cross-feature API integration.
WBS-16.1	L	0.65	Multi-service integration, automated tests and regression fixes.
WBS-16.2	M	0.30	Set up GitHub Actions, configuration and a hosted environment.
WBS-16.3	L	0.65	Run MVP acceptance cases and fix blocking demo defects.
WBS-17.1	M	0.30	Architecture, schemas, API and setup/deployment write-up.
WBS-17.2	S	0.15	Short student/admin usage guide and demo walkthrough.
TOTAL	**—**	**15.10**	**Sum of all WBS activity estimates; excludes contingency.**

Estimation method:
For consistency, size classes are converted to baseline effort as follows: T = 0.10, S = 0.15, M = 0.30, L = 0.65, XL = 1.10 person-weeks. Estimates include implementation, task-level checks and basic review. Integration, CI/CD and final acceptance testing are estimated separately under WBS-16.x. Unplanned third-party approval delays, major scope changes and an optional semantic-matching enhancement are not included. These are provisional planning assumptions to be reviewed against actual team availability.

Effort versus duration:
Total estimated effort is 15.10 person-weeks (approximately 604 person-hours). This is not the elapsed project duration. For example, a 0.65 person-week activity performed by two available developers at 0.5 FTE each could take approximately 0.65 calendar weeks, assuming parallel work is feasible and prerequisites are met. The six-person team's actual availability, dependencies and critical path will be checked in Sections 4 and 5. An exactly equal allocation would be approximately 2.52 person-weeks per member; the proposed Section 4 assignments range from 2.35 to 2.65 person-weeks. The Project Charter sets the codebase deadline at December 27, 2026 and the demo on December 30, 2026; schedule feasibility is not confirmed by effort estimates alone.

4. RESOURCES

4.1 Team Structure and Member Roles

The original Project Charter lists five students. This revised six-member plan adds Yeşim Gündoğdu. All six student IDs are confirmed in the cover and team table. Yeşim is assigned the technical role of Academic Rules & Graduation Planner Developer and the management role of Curriculum & Academic Rules Coordinator, reflecting ownership of WBS-8.1, 8.2, 9.1, 9.2 and 10.2. Effort and availability estimates remain planning assumptions subject to team review.

Member	Student ID	Role(s)	Main responsibilities
Emre Erbek	150230205	Backend & API Developer; Project Manager	API/database architecture, persistence, admin reference maintenance, permissions review, architecture documentation and team coordination.
Aykut Engür	150230215	Authentication & Student Profile Developer; Configuration Manager	Registration, JWT/roles, student profiles, academic source configuration, conversation context and dashboard data APIs.
Efe Sürücü	150240225	AI/RAG & Knowledge Base Developer; Data Quality Coordinator	Document indexing, RAG pipeline, citation handling, chatbot evaluation and AI data-quality assurance.
Konuralp Polat	150230212	Frontend & Dashboard Developer; UI/UX Coordinator	Responsive frontend, dashboard, faculty-match explanations, API integration and frontend component tests.
Samet Sav	150220209	QA & DevOps Engineer; Testing & Release Manager	Faculty metadata normalization, cross-component integration, acceptance testing and release/ demo validation.
Yeşim Gündoğdu	150220217	Academic Rules & Graduation Planner Developer; Curriculum & Academic Rules Coordinator	Curriculum modeling and validation, prerequisite-rule graph, course selection guidance, what-if scenarios, graduation progress calculation, and coordination of academic-rule consistency with Samet and other developers.

4.2 Member-to-Activity Assignment

Each of the 37 Section 2 WBS activities has one primary owner. Tasks are distributed by estimated effort, not raw task count, because RAG and integration activities are substantially larger than small configuration tasks. FTE indicates an assumed commitment while an activity is active; it does not change the Section 3 effort estimate.

WBS activity ID	Assigned member	Responsibility	Allocation to activity (FTE)
WBS-1.1	Emre Erbek	Design database schema	0.55
WBS-1.2	Emre Erbek	Build core API and persistence	0.55
WBS-2.1	Aykut Engür	Register users securely	0.55
WBS-2.2	Aykut Engür	Implement JWT login	0.55
WBS-3.1	Aykut Engür	Define access roles	0.55
WBS-3.2	Aykut Engür	Enforce protected access	0.55
WBS-4.1	Aykut Engür	Implement academic profile	0.55
WBS-4.2	Emre Erbek	Record completed courses	0.55
WBS-5.1	Aykut Engür	Curate official ITU sources	0.55
WBS-5.2	Efe Sürücü	Index academic documents	0.55
WBS-5.3	Aykut Engür	Implement document updates	0.55
WBS-6.1	Efe Sürücü	Build grounded RAG pipeline	0.55
WBS-6.2	Efe Sürücü	Cite sources and handle gaps	0.55
WBS-6.3	Efe Sürücü	Evaluate chatbot responses	0.55
WBS-7.1	Emre Erbek	Persist chat sessions	0.55
WBS-7.2	Aykut Engür	Restore conversation context	0.55
WBS-8.1	Yeşim Gündoğdu	Model supported curriculum	0.55
WBS-8.2	Yeşim Gündoğdu	Build prerequisite rule graph	0.55
WBS-9.1	Yeşim Gündoğdu	Generate course guidance	0.55
WBS-9.2	Yeşim Gündoğdu	Simulate academic scenarios	0.55
WBS-10.1	Samet Sav	Define graduation criteria	0.55
WBS-10.2	Yeşim Gündoğdu	Calculate completion gaps	0.55
WBS-11.1	Efe Sürücü	Curate faculty profiles	0.55
WBS-11.2	Samet Sav	Normalize faculty metadata	0.55
WBS-12.1	Konuralp Polat	Implement matching and ranking	0.55
WBS-12.2	Konuralp Polat	Explain recommendations	0.55

WBS-13.1	Konuralp Polat	Design student dashboard	0.55
WBS-13.2	Aykut Engür	Connect dashboard data	0.55
WBS-14.1	Emre Erbek	Build admin management views	0.55
WBS-14.2	Emre Erbek	Validate administrator actions	0.55
WBS-15.1	Konuralp Polat	Build core responsive pages	0.55
WBS-15.2	Konuralp Polat	Connect frontend to APIs	0.55
WBS-16.1	Samet Sav	Integrate and test components	0.65
WBS-16.2	Samet Sav	Configure CI and hosting	0.55
WBS-16.3	Samet Sav	Run MVP acceptance tests	0.65
WBS-17.1	Emre Erbek	Document architecture and APIs	0.55
WBS-17.2	Samet Sav	Prepare user instructions	0.55

Workload balance summary:
The 37 assignments total 15.10 person-weeks (604 hours). The six estimated workloads range from 2.35 to 2.65 person-weeks, with a spread of 0.30 person-weeks (12 hours). This reflects a relatively balanced allocation based on task complexity and individual responsibilities. Weekly availability and ownership require team confirmation.

Member	**Assigned activities**	**Estimated effort (person-weeks)**
Emre Erbek	7	2.45
Aykut Engür	9	2.55
Efe Sürücü	5	2.65
Konuralp Polat	5	2.55
Samet Sav	6	2.35
Yeşim Gündoğdu	5	2.55

Availability or allocation constraints:
Schedule assumptions: The working calendar includes Saturdays and Sundays; each planning week is seven consecutive calendar days (Monday–Sunday), and weekly effort is spread across that period. Most activities use 0.55 FTE (about 22 hours per week); Samet Sav uses 0.65 FTE (about 26 hours per week) only on WBS-16.1 and WBS-16.3 to finish acceptance testing by December 20. The team’s peak concurrent allocation is 3.30 FTE, rising to 3.40 FTE during accelerated QA. The 15.10 person-week effort does not include writing the separate university reports. Four early handoffs are coordination gates, not finish-to-start edges (Section 5.1). No primary owner has concurrent active WBS tasks.

AI-assisted contributions and error reporting (planned):
If AI coding assistants or AI agents are used, the team will maintain a shared contribution log in GitHub Projects and link each entry to the relevant issue, pull request, and WBS activity. Each record will state the date, responsible student, AI tool/model, task or prompt purpose, generated output or changed files, estimated AI usage/effort, human review and rework time, tests performed, errors introduced, detection method, and corrective action/commit. Error

categories will include incorrect logic, insecure behavior, API/interface mismatches, and unsupported or fabricated academic citations. A student remains accountable for reviewing and testing AI output before merging it. Human review/rework will be recorded as human effort and compared with the existing 604-hour software baseline; AI effort will be reported separately and will not be counted as student person-weeks. No completed AI contribution or measured effort is asserted in advance: the log will capture actual use during development.

5. SCHEDULE

5.1 PERT Chart — Activity Sequence

Planning baseline: November 3–December 22, 2026 is the active implementation and Software Test Report window; the final codebase is due December 27 and demonstration December 30. The calendar includes weekends; standard effort capacity is 0.55 FTE (22 hours spread over seven days), with 0.65 FTE (26 hours over seven days) for WBS-16.1 and WBS-16.3. Immediate predecessors in the matrix below are finish-to-start (FS). Four partial handoffs are described as non-FS coordination gates, with final acceptance of the input required before the successor is completed: WBS-1.2→2.2 (auth API contract), WBS-1.2→5.2 (DB/indexing interface), WBS-4.2→10.2 (completed-course test data), and WBS-6.2→15.2 (citation-response schema). For the four pairs, early work uses agreed mock data or interface contracts; final real integration checks occur when the predecessor is ready. These gates are shown separately from solid FS dependencies in the PERT graphic.

PERT network — principal finish-to-start branches (all FS predecessors appear in the table; dashed edges in the figure indicate interim synchronization, not FS):

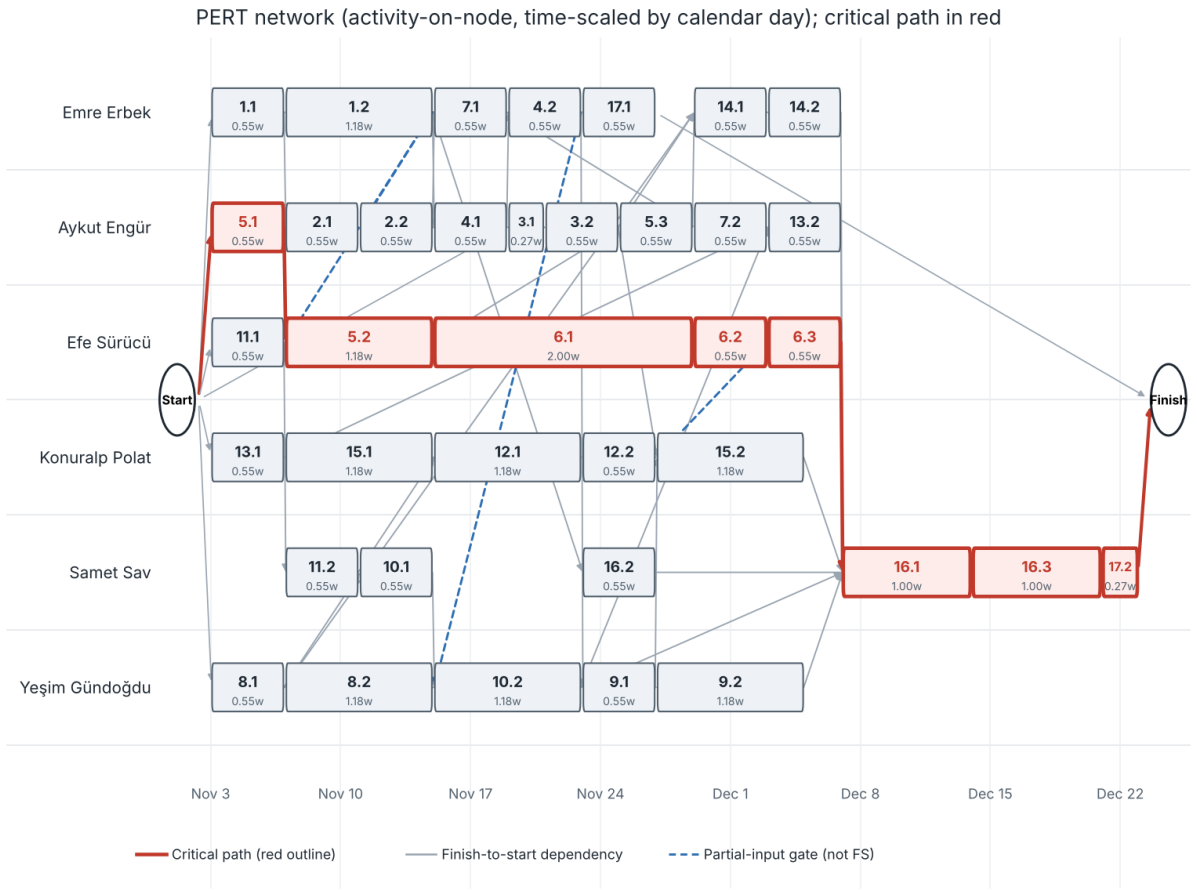
START → WBS-5.1 → WBS-5.2 → WBS-6.1 → WBS-6.2 → WBS-6.3 → WBS-16.1 → WBS-16.3 → WBS-17.2 → FINISH

START → WBS-8.1 → WBS-8.2 → WBS-9.1 → WBS-9.2 → WBS-16.1

START → WBS-11.1 → WBS-11.2 → WBS-12.1 → WBS-12.2 → WBS-15.2

START → WBS-13.1 → WBS-15.1 → WBS-15.2

START → WBS-1.1 → WBS-1.2 → WBS-16.2 → WBS-16.3



Activity predecessors and estimated elapsed durations (calendar weeks): One person-week is 40 hours of effort; a calendar week is seven consecutive days including Saturdays and Sundays. Elapsed duration is estimated as person-weeks of effort divided by the assigned FTE; Gantt starts/finishes use inclusive whole calendar dates rounded from those estimates. Calendar dates are rounded to whole days. For the eight 1.18-week activities (WBS-1.2, 5.2, 8.2, 9.2, 10.2, 12.1, 15.1 and 15.2) the eight scheduled days give 0.63 person-weeks of capacity at 0.55 FTE against 0.65 person-weeks of effort; the shortfall of under one hour per activity is absorbed within the owner's weekly allocation.

WBS activity	**Immediate predecessor(s)**	Duration (calendar weeks)	**Primary owner**
WBS-1.1	START	0.55	Emre Erbek
WBS-1.2	WBS-1.1	1.18	Emre Erbek
WBS-2.1	WBS-1.1	0.55	Aykut Engür
WBS-2.2	WBS-2.1	0.55	Aykut Engür
WBS-3.1	START	0.27	Aykut Engür
WBS-3.2	WBS-3.1, WBS-2.2	0.55	Aykut Engür
WBS-4.1	WBS-1.2, WBS-2.2	0.55	Aykut Engür
WBS-4.2	WBS-4.1	0.55	Emre Erbek

WBS-5.1	START	0.55	Aykut Engür
WBS-5.2	WBS-5.1	1.18	Efe Sürücü
WBS-5.3	WBS-5.2, WBS-3.2	0.55	Aykut Engür
WBS-6.1	WBS-5.2	2.00	Efe Sürücü
WBS-6.2	WBS-6.1	0.55	Efe Sürücü
WBS-6.3	WBS-6.2	0.55	Efe Sürücü
WBS-7.1	WBS-1.2, WBS-2.2	0.55	Emre Erbek
WBS-7.2	WBS-7.1	0.55	Aykut Engür
WBS-8.1	START	0.55	Yeşim Gündoğdu
WBS-8.2	WBS-8.1	1.18	Yeşim Gündoğdu
WBS-9.1	WBS-8.2, WBS-4.2	0.55	Yeşim Gündoğdu
WBS-9.2	WBS-9.1	1.18	Yeşim Gündoğdu
WBS-10.1	WBS-8.1	0.55	Samet Sav
WBS-10.2	WBS-10.1, WBS-8.2	1.18	Yeşim Gündoğdu
WBS-11.1	START	0.55	Efe Sürücü
WBS-11.2	WBS-11.1	0.55	Samet Sav
WBS-12.1	WBS-11.2	1.18	Konuralp Polat
WBS-12.2	WBS-12.1	0.55	Konuralp Polat
WBS-13.1	START	0.55	Konuralp Polat
WBS-13.2	WBS-13.1, WBS-7.2, WBS-10.2	0.55	Aykut Engür
WBS-14.1	WBS-5.3, WBS-8.1, WBS-3.2	0.55	Emre Erbek
WBS-14.2	WBS-14.1	0.55	Emre Erbek
WBS-15.1	WBS-13.1	1.18	Konuralp Polat
WBS-15.2	WBS-15.1, WBS-9.1, WBS-12.2, WBS-3.2	1.18	Konuralp Polat
WBS-16.1	WBS-15.2, WBS-13.2, WBS-14.2, WBS-6.3, WBS-9.2, WBS-10.2	1.00	Samet Sav
WBS-16.2	WBS-1.2	0.55	Samet Sav
WBS-16.3	WBS-16.1, WBS-16.2	1.00	Samet Sav
WBS-17.1	WBS-1.2	0.55	Emre Erbek
WBS-17.2	WBS-16.3	0.27	Samet Sav

PERT notes: Solid arrows show finish-to-start precedence. Dashed arrows mark four explicitly planned partial-input synchronization gates; they must be verified before the

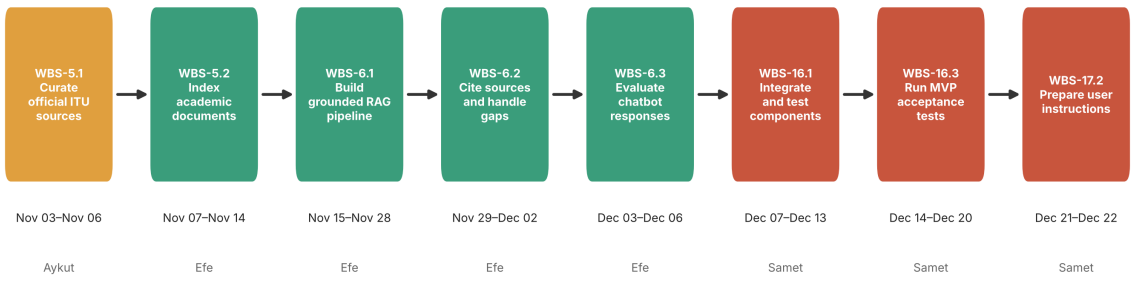
receiving task is signed off. These are not additional FS predecessors and therefore do not contradict overlapping dates in the Gantt. The source/LLM dependency chain determines the December 22 modeled finish. Report-writing deadlines (November 10 and 24, December 8 and 22) are separate academic deliverables and are not included in the 15.10 person-week implementation estimate. Real ITU SSO and native mobile work are excluded.

5.2 Critical Path and Total Project Duration

Scheduled controlling path (resource-leveled, finish-to-start; inclusive seven-day calendar dates (weekends included)): WBS-5.1 → WBS-5.2 → WBS-6.1 → WBS-6.2 → WBS-6.3 → WBS-16.1 → WBS-16.3 → WBS-17.2. These eight activities run continuously from November 3 through December 22, 2026 (50 inclusive calendar days, approximately 7.14 weeks). The four separately stated partial-input gates do not extend this controlling FS chain.

Critical-chain cross-check: WBS-5.1 (Nov 3–6) → 5.2 (Nov 7–14) → 6.1 (Nov 15–28) → 6.2 (Nov 29–Dec 2) → 6.3 (Dec 3–6) → 16.1 (Dec 7–13) → 16.3 (Dec 14–20) → 17.2 (Dec 21–22). All other FS dependencies finish before their successor starts, and no primary assignee is double-booked.

Critical chain: each step starts after the previous one finishes



Resource-leveled critical-chain activity table:

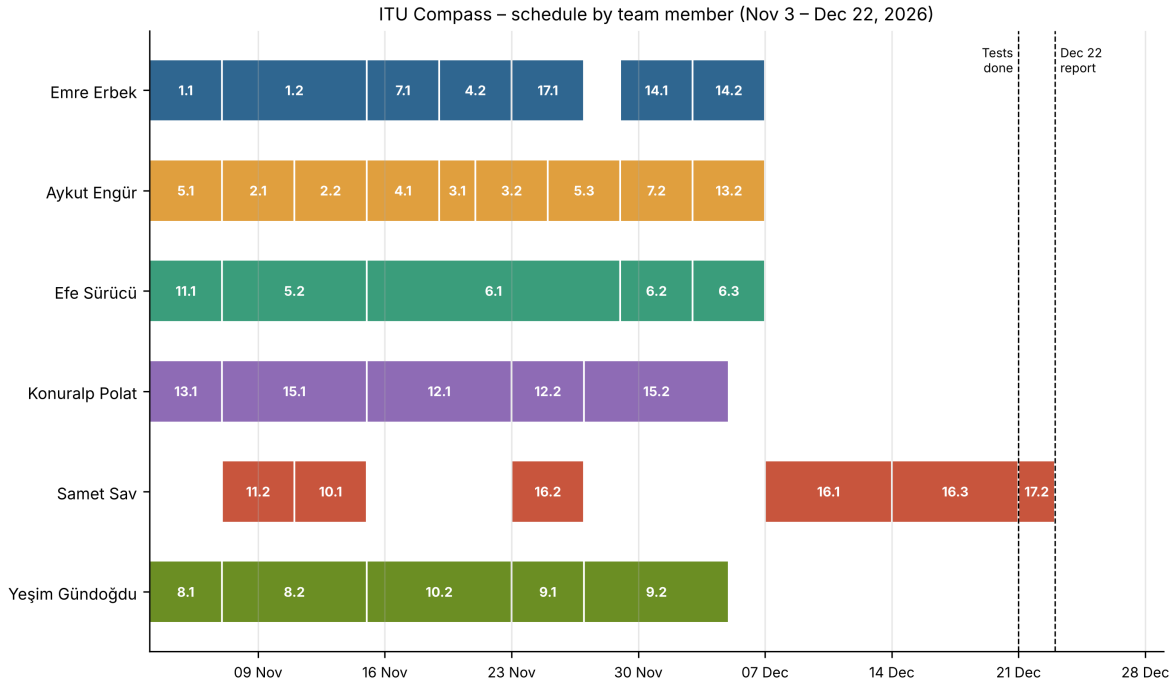
Critical WBS activity	Duration (calendar weeks)	**Constraint driving criticality**
WBS-5.1	0.55	Starting official-source curation; handoff required for indexing.
WBS-5.2	1.18	Follows WBS-5.1; validated source collection and index are available.
WBS-6.1	2.00	Follows WBS-5.2; retrieval-backed RAG generation.
WBS-6.2	0.55	Follows WBS-6.1; source citations and confidence fallbacks.

WBS-6.3	0.55	Follows WBS-6.2; question evaluation and citation review.
WBS-16.1	1.00	Follows WBS-6.3 and all other integrated component prerequisites.
WBS-16.3	1.00	Follows WBS-16.1 and the CI/hosting gate WBS-16.2.
WBS-17.2	0.27	Follows WBS-16.3; user instructions completed by Dec 22.

Critical-path explanation: The RAG/source chain determines the project finish. WBS-6.1 (November 15–28) is the longest single calendar activity. Citation completion and chatbot evaluation (WBS-6.2/6.3) run November 29–December 6, followed by integration tests December 7–13 and MVP acceptance tests December 14–20. Test evidence is therefore available December 20, giving December 21 for Software Test Report finalization before the December 22 submission. User instructions finish December 22. The final codebase submission on December 27 and the December 30 demo retain separate deadlines. Four documented partial handoffs, accelerated QA effort and agreed scope constraints are essential to meeting this compressed schedule; delays on the controlling path have negligible tolerance. Under resource leveling, two further chains also have zero slack: Aykut Engür's sequence WBS-5.1 → 2.1 → 2.2 → 4.1 → 3.1 → 3.2 → 5.3 → 7.2 → 13.2 and the admin branch WBS-5.3 → 14.1 → 14.2, both finishing December 6 and feeding WBS-16.1. All three chains are therefore monitored as critical.

5.3 Gantt Chart — Project Timeline

Gantt schedule basis: The detailed dates are inclusive calendar dates from November 3 to December 22, 2026; Saturdays and Sundays count as scheduled working days. Durations in the PERT and Gantt tables are elapsed calendar weeks (seven days), rounded to whole calendar-day start and finish dates. Standard active capacity is 0.55 FTE, increasing to 0.65 FTE for Samet on WBS-16.1 and WBS-16.3 only. No individual is double-booked. The four partial-input gates are explicitly documented in Section 5.1; all other PERT predecessors are strict finish-to-start. Software Test Report evidence is ready December 20, the report is finalized December 21 for the December 22 submission, and the codebase is due December 27.



Gantt-style workstream windows (cells list active WBS IDs; WBS- prefix is omitted):

Workstream / member	Nov3–10	Nov11–18	Nov19–26	Nov27–Dec4	Dec5–13	Dec14–22
Emre Erbek	1.1, 1.2	1.2, 7.1	4.2, 17.1	14.1, 14.2	14.2	—
Aykut Engür	5.1, 2.1	2.2, 4.1	3.1, 3.2, 5.3	5.3, 7.2, 13.2	13.2	—
Efe Sürücü	11.1, 5.2	5.2, 6.1	6.1	6.1, 6.2, 6.3	6.3	—
Konuralp Polat	13.1, 15.1	15.1, 12.1	12.1, 12.2	15.2	—	—
Samet Sav	11.2	10.1	16.2	—	16.1	16.3, 17.2
Yeşim Gündoğdu	8.1, 8.2	8.2, 10.2	10.2, 9.1	9.2	—	—

Detailed WBS activity schedule (planned start and finish, 2026):

WBS activity	Primary owner	Start date	Finish date	Duration (calendar weeks)
WBS-1.1 Design database schema	Emre Erbek	2026-11-03	2026-11-06	0.55
WBS-1.2 Build core API and persistence	Emre Erbek	2026-11-07	2026-11-14	1.18
WBS-2.1 Register users securely	Aykut Engür	2026-11-07	2026-11-10	0.55

WBS-2.2 Implement JWT login	Aykut Engür	2026-11-11	2026-11-14	0.55
WBS-3.1 Define access roles	Aykut Engür	2026-11-19	2026-11-20	0.27
WBS-3.2 Enforce protected access	Aykut Engür	2026-11-21	2026-11-24	0.55
WBS-4.1 Implement academic profile	Aykut Engür	2026-11-15	2026-11-18	0.55
WBS-4.2 Record completed courses	Emre Erbek	2026-11-19	2026-11-22	0.55
WBS-5.1 Curate official ITU sources	Aykut Engür	2026-11-03	2026-11-06	0.55
WBS-5.2 Index academic documents	Efe Sürücü	2026-11-07	2026-11-14	1.18
WBS-5.3 Implement document updates	Aykut Engür	2026-11-25	2026-11-28	0.55
WBS-6.1 Build grounded RAG pipeline	Efe Sürücü	2026-11-15	2026-11-28	2.00
WBS-6.2 Cite sources and handle gaps	Efe Sürücü	2026-11-29	2026-12-02	0.55
WBS-6.3 Evaluate chatbot responses	Efe Sürücü	2026-12-03	2026-12-06	0.55
WBS-7.1 Persist chat sessions	Emre Erbek	2026-11-15	2026-11-18	0.55
WBS-7.2 Restore conversation context	Aykut Engür	2026-11-29	2026-12-02	0.55
WBS-8.1 Model supported curriculum	Yeşim Gündoğdu	2026-11-03	2026-11-06	0.55
WBS-8.2 Build prerequisite rule graph	Yeşim Gündoğdu	2026-11-07	2026-11-14	1.18
WBS-9.1 Generate course guidance	Yeşim Gündoğdu	2026-11-23	2026-11-26	0.55
WBS-9.2 Simulate academic scenarios	Yeşim Gündoğdu	2026-11-27	2026-12-04	1.18
WBS-10.1 Define graduation criteria	Samet Sav	2026-11-11	2026-11-14	0.55
WBS-10.2 Calculate completion gaps	Yeşim Gündoğdu	2026-11-15	2026-11-22	1.18
WBS-11.1 Curate faculty profiles	Efe Sürücü	2026-11-03	2026-11-06	0.55
WBS-11.2 Normalize faculty metadata	Samet Sav	2026-11-07	2026-11-10	0.55
WBS-12.1 Implement matching and ranking	Konuralp Polat	2026-11-15	2026-11-22	1.18

WBS-12.2 Explain recommendations	Konuralp Polat	2026-11-23	2026-11-26	0.55
WBS-13.1 Design student dashboard	Konuralp Polat	2026-11-03	2026-11-06	0.55
WBS-13.2 Connect dashboard data	Aykut Engür	2026-12-03	2026-12-06	0.55
WBS-14.1 Build admin management views	Emre Erbek	2026-11-29	2026-12-02	0.55
WBS-14.2 Validate administrator actions	Emre Erbek	2026-12-03	2026-12-06	0.55
WBS-15.1 Build core responsive pages	Konuralp Polat	2026-11-07	2026-11-14	1.18
WBS-15.2 Connect frontend to APIs	Konuralp Polat	2026-11-27	2026-12-04	1.18
WBS-16.1 Integrate and test components	Samet Sav	2026-12-07	2026-12-13	1.00
WBS-16.2 Configure CI and hosting	Samet Sav	2026-11-23	2026-11-26	0.55
WBS-16.3 Run MVP acceptance tests	Samet Sav	2026-12-14	2026-12-20	1.00
WBS-17.1 Document architecture and APIs	Emre Erbek	2026-11-23	2026-11-26	0.55
WBS-17.2 Prepare user instructions	Samet Sav	2026-12-21	2026-12-22	0.27

Academic submission checkpoints (separate from software-product WBS deliverables): Project Plan — November 3, 2026 at 23:59 (Ninova deadline); Requirements Specification — November 10, 2026; Requirements Analysis — November 24, 2026; Software Design — December 8, 2026; Software Test Report — December 22, 2026; Final Codebase — December 27, 2026; Demo & Presentation — December 30, 2026. The team's academic submissions must be coordinated alongside the implementation timeline.

Academic report preparation and capacity allocation (proposed):
The academic reports are separate from the eight software-product deliverables and their writing hours are not included in the 15.10 person-week (604-hour) implementation baseline. Proposed coordinating owners are Emre for Requirements Specification (November 10), Yeşim for Requirements Analysis (November 24), Emre for Software Design (December 8), and Samet for the Software Test Report (December 22). The coordinator will target a reviewable draft at least 48 hours before each deadline, with relevant technical owners providing test/design evidence and completing peer review at least 24 hours before submission. As a provisional reporting reserve, budget 4–6 hours for each report coordinator plus 1–2 hours for each contributing reviewer; actual report hours and upload responsibility will be recorded separately in GitHub Projects. The 22 hours per person per seven-day week assumed for active WBS work are not free extra capacity: if report preparation uses any of those hours, effective WBS availability must be reduced and the Gantt, handoffs, and critical path rechecked. These allocations are planning assumptions and require team confirmation.

Schedule consistency check: All 37 WBS activities, 17 functional tasks and eight product deliverables are represented. Implementation effort remains 15.10 person-weeks (604 hours); adjusted owners, the 5.1 FS network, the critical-chain diagram and the Gantt use identical dates. The controlling chain finishes December 22, with acceptance testing completed December 20. Final codebase and demo deadlines remain December 27 and 30. Academic report preparation is scheduled alongside development but excluded from this software effort baseline. The team must confirm all task allocations, availability, interim synchronization gates and upload the Project Plan before the confirmed November 3, 23:59 deadline.

6. RISKS

6.1 Risk Impact / Probability Chart

Probability and impact are scored on a proposed 1 (very low) to 5 (very high) ordinal scale. Risk score = Probability × Impact. The register is sorted by ascending probability and, within each probability level, ascending impact as required by the course template. Scores are planning judgments rather than measured event frequencies. All affected activity IDs refer to Section 2 WBS items.

Risk register (ascending probability, then impact):

Risk ID	Risk description	Probability (1–5)	Impact (1–5)	Risk score	Affected WBS ID(s)
R-01	Public faculty profile data is incomplete or cannot be reused as planned.	2	3	6	WBS-11.1, WBS-11.2, WBS-12.1
R-02	Authentication or authorization defects expose student profiles or chat histories.	2	4	8	WBS-2.2, WBS-3.2, WBS-7.2
R-03	Cloud deployment fails because of configuration, secrets, or hosting restrictions.	2	4	8	WBS-16.2, WBS-16.3
R-04	LLM API rate limits, downtime, or usage costs interrupt chatbot features.	3	3	9	WBS-6.1, WBS-6.3
R-05	Official ITU curricula, academic rules, or source documents are outdated or inconsistent.	3	4	12	WBS-5.1, WBS-8.1, WBS-8.2, WBS-10.1

R-06	Faculty-topic matching produces irrelevant or poorly explained recommendations.	3	4	12	WBS-11.2, WBS-12.1, WBS-12.2
R-07	RAG chatbot produces unsupported or misleading academic guidance.	3	5	15	WBS-5.2, WBS-6.1, WBS-6.2, WBS-6.3
R-08	Frontend-backend contract mismatches delay integrated user journeys.	4	3	12	WBS-15.2, WBS-16.1
R-09	Six students cannot consistently maintain the assumed 0.55 FTE during coursework or exams.	4	4	16	WBS-15.2, WBS-16.1, WBS-16.3
R-10	Additional scope requests expand the MVP beyond its approved web and data boundaries.	4	4	16	WBS-12.1, WBS-15.1, WBS-15.2, WBS-16.3
R-11	Critical-chain delays consume the small buffer before the December 27 codebase deadline.	4	5	20	WBS-6.1, WBS-15.2, WBS-16.1, WBS-16.3

RISK MATRIX (impact rows × probability columns; cells show risk IDs):

Impact ↓ / Probability →	**1 Very low**	**2 Low**	**3 Medium**	**4 High**	**5 Very high**
1 Very low	—	—	—	—	—
2 Low	—	—	—	—	—
3 Medium	—	R-01	R-04	R-08	—
4 High	—	R-02, R-03	R-05, R-06	R-09, R-10	—
5 Very high	—	—	R-07	R-11	—

Priority interpretation: Risks with scores of 15 or above require the most immediate attention, followed by score 12 risks. This ranking supplements—rather than replaces—critical-path and security assessments. The 0.55 FTE weekly availability assumption and the compressed November 3 – December 22 window are especially important schedule uncertainties.

6.2 Risk Mitigation / Aversion Plan

Each risk is linked to implementable WBS activities and assigned to one accountable team member. Mitigation actions aim to prevent or reduce the risk; contingency actions describe the response if it happens. These measures keep the MVP within its approved scope: an ITU Compass web app using demo authentication, a limited set of official ITU academic sources and curated faculty profiles.

Risk mitigation and contingency register:

Risk ID	Related WBS activity ID(s)	Mitigation / avoidance strategy	Responsible member	Contingency if risk occurs
R-01	WBS-11.1, WBS-11.2, WBS-12.1	Use a small manually curated set of permitted public ITU profiles; retain source URLs and verify reuse conditions.	Samet Sav	Limit matching to verified profiles and clearly state incomplete coverage; skip unavailable fields.
R-02	WBS-2.2, WBS-3.2, WBS-7.2	Hash passwords, validate JWT expiry and roles, isolate records by user, and run negative access tests.	Aykut Engür	Disable affected endpoints, revoke demo tokens and repair access controls before reopening.
R-03	WBS-16.2, WBS-16.3	Provision deployment early, verify environment variables, protect API keys, and test CI/CD on staging.	Samet Sav	Use an approved alternate host or a reproducible local Docker demo while restoring cloud access.
R-04	WBS-6.1, WBS-6.3	Monitor token usage, set spending limits and timeouts, cap request size, and cache safe test queries.	Efe Sürücü	Show a service-unavailable notice and offer document links and rule-based functions without AI.
R-05	WBS-5.1, WBS-8.1, WBS-8.2, WBS-10.1	Record academic year, faculty/department, publication date and official URL; manually review sample rules.	Yeşim Gündoğdu	Restrict calculations to validated curriculum versions and flag unresolved cases for official verification.
R-06	WBS-11.2, WBS-12.1, WBS-12.2	Start with a transparent keyword baseline, evaluate representative topics, and check source-linked explanations.	Samet Sav	Offer filterable faculty profiles and keyword search instead of unreliable ranked recommendations.

R-07	WBS-5.2, WBS-6.1, WBS-6.2, WBS-6.3	Require retrieved official evidence, page-linked citations, low-evidence refusal, and human evaluation sets.	Efe Sürücü	Disable unsupported answers and show verified source excerpts plus an official-adviser disclaimer.
R-08	WBS-15.2, WBS-16.1	Agree OpenAPI schemas early, test mocked and real responses, and integrate incrementally.	Konuralp Polat	Freeze API shapes and prioritize functioning end-to-end core flows over secondary dashboard widgets.
R-09	WBS-15.2, WBS-16.1, WBS-16.3	Confirm weekly availability, cap concurrent assignments, track actual hours and review schedule weekly.	Emre Erbek	Reassign blockers, shorten nonessential UI polish, and rebaseline milestone dates with the instructor.
R-10	WBS-12.1, WBS-15.1, WBS-15.2, WBS-16.3	Keep an MVP acceptance checklist; request Product Owner approval before changing features or datasets.	Emre Erbek	Defer optional enhancements such as semantic matching or SSO integration to a documented later phase.
R-11	WBS-6.1, WBS-15.2, WBS-16.1, WBS-16.3	Monitor critical tasks twice weekly, validate current work status and progress against the approved schedule, and establish integration checkpoints.	Emre Erbek	Deliver a stable reduced MVP with RAG citations, prerequisite checks and essential web screens; disclose deferred features.

Monitoring and escalation: Review open risks and actual activity progress in a short weekly team meeting and record decisions in GitHub Projects. Escalate a high-score risk promptly to the project manager and relevant owner. If 0.55 FTE capacity is not achieved, if critical-path activities slip, or if source data is unavailable, replan the WBS/Gantt and prioritize the citation-backed academic assistant, rule checks and web demonstration. Do not claim actual ITU SSO connectivity, official advising authority, or supervisor availability. Dates, probability and impact assessments are proposals for team confirmation.